

she suggested that since the `total_num_tickets` is only printed when the application exits at the end of the day, you can parallelise this counter so that the contention on the single shared counter is removed. You then came up with the following version of code:

```
int num_tickets_per_thread [N]; //N is the number of threads in your application
```

```
incrementTicketCounter()
{
    int thread_id = pthread_self();
    num_total_tickets[thread_id]++;
}

void PrintTotalTickets()
{
    //only the main thread calls this function, so no locking needed
    for (int i = 0; i < N; i++)
        num_total_tickets += Num_tickets_per_thread[i];
    printf ("%d", num_total_tickets);
}
```

Note that the API `pthread_self` gets the ID of the thread calling this function. Does this code have any performance problems? At first glance, it does not appear so. All the threads update their own array element. Hence no synchronisation is needed between them. `PrintTotalTickets` is called only by the main thread to print the total number of tickets when the program finishes. So this function also does not need any synchronisation to access the `num_tickets_per_thread` array. Therefore, where is the performance bottleneck?

Remember that your multi-threaded application runs on a multi-processor system. Each processor has its own cache and the cache coherence is enforced through hardware. Cache coherence ensures that different copies of the same memory location kept in each of the processor's local caches are consistent with each other. When a processor needs to modify data in its cache that is also present in other processors' caches, it first needs to invalidate the copies present in other caches. This is known as cache line invalidation.

Now consider the update of each array element

`num_tickets_per_thread[thread_id]` by each thread. Assuming a cache line of size 64 bytes, we see that 16 elements of the array can fit in the same cache line. Now, whenever any of these threads updates an array element, it will invalidate the copies present in all other processors' cache. This will result in that particular cache line ping-ponging across all caches of the processors that access this array. This problem is known as false sharing. A processor updating the array element it owns, ends up invalidating the other processor's cache copies because of their co-location in the same cache line.

Unlike true sharing where the same datum is accessed by multiple threads, in this case, different threads access different data items. Because of their co-location in the same cache line, the line gets migrated across the different processor caches. This phenomenon is known as false sharing. This will result in an increased number of cache misses and will show up as an increase in the execution time of the application. Also, as the number of threads in the application increases, the performance becomes poorer as the cache line now ping-pongs to more processor caches, causing a severe scalability issue.

Takeaway problem for this month

Now how can we work around the false cache line-sharing problem in our example code snippet? We need to pad each array element so that each element is present in a separate cache line. This will avoid the cache line false sharing, but will come at the expense of an increased space allotted to the array. Our takeaway problem this month is to modify our sample code so as to eliminate false sharing. Please do send me your solutions to this.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.

THE COMPLETE MAGAZINE ON OPEN SOURCE

LINUX

BETA
ForYou

Your favourite Linux Magazine is
now on the Web, too.

LinuxForU.com

Follow us on Twitter @LinuxForYou